

relatively large errors when determined by the routine `hqr` (§11.7). Had we merely inspected the matrix beforehand, we could have determined the isolated eigenvalue exactly and then deleted the corresponding row and column from the matrix. You should consider whether such a pre-inspection might be useful in your application. (For symmetric matrices, the routines we gave will determine isolated eigenvalues accurately in all cases.)

The routine `balance` keeps track of the scale factors used in the balancing. If you are computing eigenvectors as well as eigenvalues, then the accumulated similarity transformation of the original matrix is undone by applying these scale factors in the routine `balbak`.

`void Unsymmeig::balance()`

`eigen_unsym.h`

Given a matrix `a[0..n-1][0..n-1]`, this routine replaces it by a balanced matrix with identical eigenvalues. A symmetric matrix is already balanced and is unaffected by this procedure.

```
{
    const Doub RADIX = numeric_limits<Doub>::radix;
    Bool done=false;
    Doub sqrdx=RADIX*RADIX;
    while (!done) {
        done=true;
        for (Int i=0;i<n;i++) {
            Doub r=0.0,c=0.0;
            for (Int j=0;j<n;j++)
                if (j != i) {
                    c += abs(a[j][i]);
                    r += abs(a[i][j]);
                }
            if (c != 0.0 && r != 0.0) {
                Doub g=r/RADIX;
                Doub f=1.0;
                Doub s=c+r;
                while (c<g) {
                    f *= RADIX;
                    c *= sqrdx;
                }
                g=r*RADIX;
                while (c>g) {
                    f /= RADIX;
                    c /= sqrdx;
                }
                if ((c+r)/f < 0.95*s) {
                    done=false;
                    g=1.0/f;
                    scale[i] *= f;
                    for (Int j=0;j<n;j++) a[i][j] *= g;
                    for (Int j=0;j<n;j++) a[j][i] *= f;
                }
            }
        }
    }
}
```

Calculate row and column norms.

If both are nonzero,

find the integer power of the machine radix that comes closest to balancing the matrix.

Apply similarity transformation.

`void Unsymmeig::balbak()`

`eigen_unsym.h`

Forms the eigenvectors of a real nonsymmetric matrix by back transforming those of the corresponding balanced matrix determined by `balance`.

```
{
    for (Int i=0;i<n;i++)
        for (Int j=0;j<n;j++)
            zz[i][j] *= scale[i];
}
```

11.6.2 Reduction to Hessenberg Form

An *upper Hessenberg* matrix has zeros everywhere below the diagonal except for the first subdiagonal row. For example, in the 6×6 case, the nonzero elements are

$$\begin{bmatrix} \times & \times & \times & \times & \times & \times \\ \times & \times & \times & \times & \times & \times \\ & \times & \times & \times & \times & \times \\ & & \times & \times & \times & \times \\ & & & \times & \times & \times \\ & & & & \times & \times \end{bmatrix}$$

By now you should be able to tell at a glance that such a structure can be achieved by a sequence of Householder transformations, each one zeroing the required elements in a column of the matrix. Householder reduction to Hessenberg form is in fact an accepted technique. An alternative, however, is a procedure analogous to Gaussian elimination with pivoting. We will use this elimination procedure since it is about a factor of two more efficient than the Householder method, and also since we want to teach you the method. It is possible to construct matrices for which the Householder reduction, being orthogonal, is stable and elimination is not, but such matrices are extremely rare in practice.

Straight Gaussian elimination is not a similarity transformation of the matrix. Accordingly, the actual elimination procedure used is slightly different. We proceed in a series of stages $r = 1, 2, \dots, N - 2$. Before the r th stage, the original matrix $\mathbf{A} \equiv \mathbf{A}_1$ has become $\mathbf{A}_r$, which is upper Hessenberg up to, but not including, row and column $r - 1$. The r th stage then consists of the following sequence of operations:

- Find the element of maximum magnitude in column $r - 1$ below the diagonal. If it is zero, skip the next two “bullets” and the stage is done. Otherwise, suppose the maximum element was in row r' .
- Interchange rows r' and r . This is the pivoting procedure. To make the permutation a similarity transformation, also interchange columns r' and r .
- For $i = r + 1, r + 2, \dots, N - 1$, compute the multiplier

$$n_{ir} \equiv \frac{a_{i,r-1}}{a_{r,r-1}}$$

Subtract n_{ir} times row r from row i . To make the elimination a similarity transformation, also add n_{ir} times column i to column r .

A total of $N - 2$ such stages are required.

When the magnitudes of the matrix elements vary over many orders, you should try to rearrange the matrix so that the largest elements are in the top left-hand corner. This reduces the roundoff error, since the reduction proceeds from left to right.

The routine `elmhes` keeps track of the permutations applied during the elimination. If you are computing eigenvectors, then the accumulated similarity transformation is applied to the eigenvectors by the routine `eltran`, which includes any necessary permutations. The operation count is about $5N^3/3$ for large N .

```
void Unsymmeig::elmhes()
```

eigen_unsym.h

Reduction to Hessenberg form by the elimination method. Replaces the real nonsymmetric matrix $a[0..n-1][0..n-1]$ by an upper Hessenberg matrix with identical eigenvalues. Recommended, but not required, is that this routine be preceded by `balance`. On output, the Hessenberg matrix is in elements $a[i][j]$ with $i \leq j+1$. Elements with $i > j+1$ are to be thought of as zero, but are returned with random values.

```
{
    for (Int m=1;m<n-1;m++) {           m is called r in the text.
        Doub x=0.0;
        Int i=m;
        for (Int j=m;j<n;j++) {         Find the pivot.
            if (abs(a[j][m-1]) > abs(x)) {
                x=a[j][m-1];
                i=j;
            }
        }
        perm[m]=i;                      Store permutation.
        if (i != m) {                   Interchange rows and columns.
            for (Int j=m-1;j<n;j++) SWAP(a[i][j],a[m][j]);
            for (Int j=0;j<n;j++) SWAP(a[j][i],a[j][m]);
        }
        if (x != 0.0) {                 Carry out the elimination.
            for (i=m+1;i<n;i++) {
                Doub y=a[i][m-1];
                if (y != 0.0) {
                    y /= x;
                    a[i][m-1]=y;
                    for (Int j=m;j<n;j++) a[i][j] -= y*a[m][j];
                    for (Int j=0;j<n;j++) a[j][m] += y*a[j][i];
                }
            }
        }
    }
}
```

```
void Unsymmeig::eltran()
```

eigen_unsym.h

This routine accumulates the stabilized elementary similarity transformations used in the reduction to upper Hessenberg form by `elmhes`. The multipliers that were used in the reduction are obtained from the lower triangle (below the subdiagonal) of a . The transformations are permuted according to the permutations stored in `perm` by `elmhes`.

```
{
    for (Int mp=n-2;mp>0;mp--) {
        for (Int k=mp+1;k<n;k++)
            zz[k][mp]=a[k][mp-1];
        Int i=perm[mp];
        if (i != mp) {
            for (Int j=mp;j<n;j++) {
                zz[mp][j]=zz[i][j];
                zz[i][j]=0.0;
            }
            zz[i][mp]=1.0;
        }
    }
}
```

CITED REFERENCES AND FURTHER READING:

- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer).[1]
- Smith, B.T., et al. 1976, *Matrix Eigensystem Routines — EISPACK Guide*, 2nd ed., vol. 6 of *Lecture Notes in Computer Science* (New York: Springer).[2]
- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §6.5.4.[3]